WEEK 8 – DATABASE RELATIONSHIP ANALYSIS USING JOINS

CITYCARE HOSPITAL MANAGEMENT SYSTEM

1. Introduction

Database Relationship Analysis using Joins is an important part of the CityCare Hospital Management System. It is used to retrieve and combine related information from multiple database tables.

The hospital maintains patient details, medicine information, patient orders, order details, and payment transactions in separate tables. SQL joins help retrieve meaningful information by establishing relationships between these tables.

In this module, INNER JOIN, LEFT JOIN, and RIGHT JOIN are implemented to analyse patient orders, medicine purchases, payment transactions, and hospital revenue.

2. Objectives

The main objectives of this module are:

- To understand relationships between multiple database tables.
- To implement INNER JOIN, LEFT JOIN, and RIGHT JOIN.
- To retrieve patient details along with their orders.
- To identify patients who have not placed orders.
- To display medicines that have not been purchased.
- To combine order and payment information.
- To analyse patient purchase history.
- To generate medicine sales and payment reports.
- To identify patients with maximum orders and highest spending.

3. Tables Used

The following tables are used in this module:

Table Name	Description
Patient	Stores patient information
Patient_Orders	Stores orders placed by patients
Order_Details	Stores medicine quantities and prices for each order
Medicine	Stores medicine information
Hospital_Payment	Stores payment transaction details

4. IMPLEMENTATION OF INNER JOIN

INNER JOIN retrieves only matching records from two or more related tables.

4.1 Display Patient Details Along With Their Orders

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    po.Order_ID,
    po.Total_Amount
FROM Patient p
INNER JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID;
```

Explanation: This query displays patient details along with their corresponding order IDs and total amounts.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      po.Order_ID,
  5      po.Total_Amount
  6  FROM Patient p
  7  INNER JOIN Patient_Orders po
  8  ON p.Patient_ID = po.Patient_ID;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
       301
Ananya Kumar
       501           13

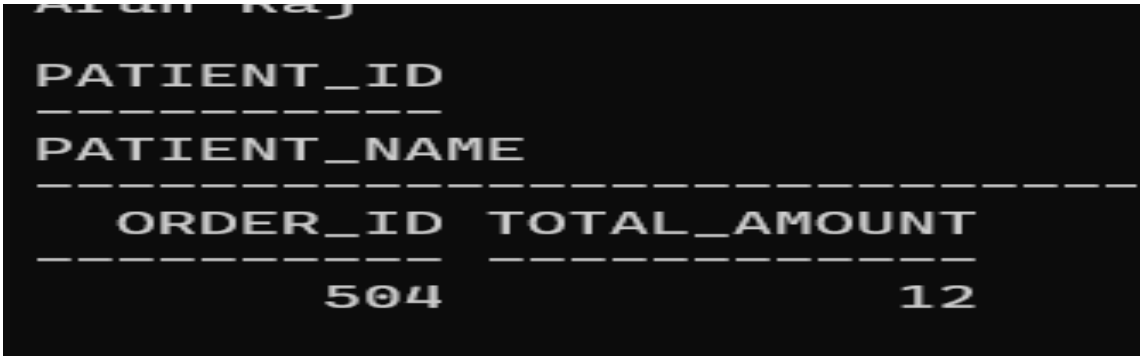
       302
Rahul Sharma
       502         10.5

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------

       303
Meena Devi
       503           10

       304
Arun Raj

PATIENT_ID
----------
```



4.2 Display Order Details With Payment Information

SQL Query:

```
SELECT
    po.Order_ID,
    po.Total_Amount,
    po.Order_Date,
    hp.Payment_Date,
    hp.Payment_Mode,
    hp.Payment_Status
FROM Patient_Orders po
INNER JOIN Hospital_Payment hp
ON po.Order_ID = hp.Order_ID;
```

Explanation: This query combines patient order information with payment details to display the payment mode and transaction status.

Output:

```
SQL> SELECT
  2       po.Order_ID,
  3       po.Total_Amount,
  4       po.Order_Date,
  5       hp.Payment_Date,
  6       hp.Payment_Mode,
  7       hp.Payment_Status
  8  FROM Patient_Orders po
  9  INNER JOIN Hospital_Payment hp
 10  ON po.Order_ID = hp.Order_ID;

  ORDER_ID TOTAL_AMOUNT ORDER_DAT PAYMENT_D PAYMENT_MODE
---------- ------------ --------- --------- --------------------
PAYMENT_STATUS
--------------------
       501           13 04-OCT-26 05-OCT-26 UPI
Success

       502         10.5 04-OCT-26 05-OCT-26 Card
Success

       503           10 04-OCT-26 05-OCT-26 Cash
Success


  ORDER_ID TOTAL_AMOUNT ORDER_DAT PAYMENT_D PAYMENT_MODE
---------- ------------ --------- --------- --------------------
PAYMENT_STATUS
--------------------
       504           12 04-OCT-26 05-OCT-26 Net Banking
Failed
```

4.3 Retrieve Medicines Purchased by Patients

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    po.Order_ID,
    m.Medicine_Name,
    od.Quantity,
    od.Price,
    od.Quantity * od.Price AS Total_Amount
FROM Patient p
INNER JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
INNER JOIN Order_Details od
ON po.Order_ID = od.Order_ID
INNER JOIN Medicine m
ON od.Medicine_ID = m.Medicine_ID;
```

Explanation: This query retrieves the medicines purchased by patients, including medicine name, quantity, price, and calculated amount.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      po.Order_ID,
  5      m.Medicine_Name,
  6      od.Quantity,
  7      od.Price,
  8      od.Quantity * od.Price AS Total_Amount
  9  FROM Patient p
 10  INNER JOIN Patient_Orders po
 11  ON p.Patient_ID = po.Patient_ID
 12  INNER JOIN Order_Details od
 13  ON po.Order_ID = od.Order_ID
 14  INNER JOIN Medicine m
 15  ON od.Medicine_ID = m.Medicine_ID;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT
---------- ---------- ------------
       301
Ananya Kumar
       501

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT
---------- ---------- ------------
```

```
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT
---------- ---------- ------------
Paracetamol
         2        2.5            5


PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT
---------- ---------- ------------
       301
Ananya Kumar
       501

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT
---------- ---------- ------------
Amoxicillin
         1          8            8


PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
```

```
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT
---------- ---------- ------------
Omeprazole
         2          5           10


PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT
---------- ---------- ------------
       304
Arun Raj
       504

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT
---------- ---------- ------------
Azithromycin
         1         12           12
```

5. IMPLEMENTATION OF LEFT JOIN

LEFT JOIN retrieves all records from the left table and matching records from the right table. If no matching record exists, NULL values are returned.

5.1 Display All Patients Including Patients Without Orders

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    po.Order_ID,
    po.Order_Date,
    po.Total_Amount,
    po.Order_Status
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
ORDER BY p.Patient_ID;
```

Explanation: This query displays all registered patients, including those who have not placed any orders.

Output:

```
SQL> SELECT
  2       p.Patient_ID,
  3       p.Patient_Name,
  4       po.Order_ID,
  5       po.Order_Date,
  6       po.Total_Amount,
  7       po.Order_Status
  8  FROM Patient p
  9  LEFT JOIN Patient_Orders po
 10  ON p.Patient_ID = po.Patient_ID
 11  ORDER BY p.Patient_ID;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------
       301
Ananya Kumar
       501 04-OCT-26           13 Delivered

       302
Rahul Sharma
       502 04-OCT-26         10.5 Pending

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------

       303
Meena Devi
       503 04-OCT-26           10 Pending

       304
Arun Raj
```

```
PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------
  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------

       504 04-OCT-26           12 Pending

       305
Kavitha S
```

5.2 Find Patients Without Any Purchases

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    p.Phone_Number
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
WHERE po.Order_ID IS NULL;
```

Explanation: This query identifies patients who do not have any records in the Patient_Orders table.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      p.Phone_Number
  5  FROM Patient p
  6  LEFT JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  WHERE po.Order_ID IS NULL;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------
PHONE_NUMBER
---------------
       305
Kavitha S
9876500015
```

5.3 Display All Medicines Including Medicines With No Sales

SQL Query:

```
SELECT
    m.Medicine_ID,
    m.Medicine_Name,
    m.Unit_Price,
    od.Order_ID,
    od.Quantity
FROM Medicine m
LEFT JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
ORDER BY m.Medicine_ID;
```

Explanation: This query displays all medicines, including medicines that have not been purchased by any patient.

Output:

```
SQL> SELECT
  2      m.Medicine_ID,
  3      m.Medicine_Name,
  4      m.Unit_Price,
  5      od.Order_ID,
  6      od.Quantity
  7  FROM Medicine m
  8  LEFT JOIN Order_Details od
  9  ON m.Medicine_ID = od.Medicine_ID
 10  ORDER BY m.Medicine_ID;

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------------------------
UNIT_PRICE   ORDER_ID   QUANTITY
---------- ---------- ----------
       201
Paracetamol
       2.5        501          2

       202
Amoxicillin
         8        501          1

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------------------------
UNIT_PRICE   ORDER_ID   QUANTITY
---------- ---------- ----------

       203
Cetirizine
       3.5        502          5

       204
Omeprazole

MEDICINE_ID
```

```
MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------------
UNIT_PRICE   ORDER_ID   QUANTITY
---------- ---------- ----------
         5        503          2

        205
Azithromycin
        12        504          1

        206

MEDICINE_ID
-----------
MEDICINE_NAME
--------------------------------------------------------------------
UNIT_PRICE   ORDER_ID   QUANTITY
---------- ---------- ----------
Ibuprofen
         4


6 rows selected.
```

6. IMPLEMENTATION OF RIGHT JOIN

RIGHT JOIN retrieves all records from the right table and matching records from the left table.

6.1 Display All Orders With Patient Information

SQL Query:

```
SELECT
    po.Order_ID,
    po.Order_Date,
    po.Total_Amount,
    po.Order_Status,
    p.Patient_ID,
    p.Patient_Name
FROM Patient p
RIGHT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
ORDER BY po.Order_ID;
```

Explanation: This query retrieves all patient orders along with the corresponding patient information.

Outpu

```
SQL> SELECT
  2      po.Order_ID,
  3      po.Order_Date,
  4      po.Total_Amount,
  5      po.Order_Status,
  6      p.Patient_ID,
  7      p.Patient_Name
  8  FROM Patient p
  9  RIGHT JOIN Patient_Orders po
 10  ON p.Patient_ID = po.Patient_ID
 11  ORDER BY po.Order_ID;

  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PATIENT_ID
---------- --------- ------------ -------------------- ----------
PATIENT_NAME
--------------------------------------------------------------------------------
       501 04-OCT-26           13 Delivered                    301
Ananya Kumar

       502 04-OCT-26         10.5 Pending                      302
Rahul Sharma

       503 04-OCT-26           10 Pending                      303
Meena Devi


  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         PATIENT_ID
---------- --------- ------------ -------------------- ----------
PATIENT_NAME
--------------------------------------------------------------------------------
       504 04-OCT-26           12 Pending                      304
Arun Raj
```

6.2 Find Orders Where Patient Details Are Missing

SQL Query:

```
SELECT
    po.Order_ID,
    po.Order_Date,
    po.Total_Amount
FROM Patient p
RIGHT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
WHERE p.Patient_ID IS NULL;
```

Explanation: This query checks for orders that do not have matching patient details. If all foreign-key relationships are valid, no records may be returned.

Output:

```
SQL> SELECT
  2      po.Order_ID,
  3      po.Order_Date,
  4      po.Total_Amount
  5  FROM Patient p
  6  RIGHT JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  WHERE p.Patient_ID IS NULL;

no rows selected
```

6.3 Display All Payment Records With Order Details

SQL Query:

```
SELECT
    hp.Payment_ID,
    hp.Payment_Date,
    hp.Payment_Mode,
    hp.Payment_Status,
    hp.Transaction_Amount,
    po.Order_ID,
    po.Order_Date,
    po.Total_Amount,
    po.Order_Status
FROM Patient_Orders po
RIGHT JOIN Hospital_Payment hp
ON po.Order_ID = hp.Order_ID
ORDER BY hp.Payment_ID;
```

Explanation: This query retrieves all payment transactions together with their corresponding order information.

Output:

```
SQL> SELECT
  2      hp.Payment_ID,
  3      hp.Payment_Date,
  4      hp.Payment_Mode,
  5      hp.Payment_Status,
  6      hp.Transaction_Amount,
  7      po.Order_ID,
  8      po.Order_Date,
  9      po.Total_Amount,
 10      po.Order_Status
 11  FROM Patient_Orders po
 12  RIGHT JOIN Hospital_Payment hp
 13  ON po.Order_ID = hp.Order_ID
 14  ORDER BY hp.Payment_ID;

PAYMENT_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT   ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------------------ ---------- --------- ------------ --------------------
       701 05-OCT-26 UPI                  Success
                13        501 04-OCT-26           13 Delivered

       702 05-OCT-26 Card                 Success
              17.5        502 04-OCT-26         10.5 Pending

       703 05-OCT-26 Cash                 Success
                10        503 04-OCT-26           10 Pending


PAYMENT_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT   ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------------------ ---------- --------- ------------ --------------------
       704 05-OCT-26 Net Banking          Failed
                12        504 04-OCT-26           12 Pending
```

7. RETRIEVE COMPLETE ORDER DETAILS

7.1 Complete Patient Order Report

SQL Query:

```
SELECT
    p.Patient_Name,
    m.Medicine_Name,
    od.Quantity,
    po.Order_Date,
    po.Total_Amount,
    hp.Payment_Status
FROM Patient p
INNER JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
INNER JOIN Order_Details od
ON po.Order_ID = od.Order_ID
INNER JOIN Medicine m
```

```
ON od.Medicine_ID = m.Medicine_ID
LEFT JOIN Hospital_Payment hp
ON po.Order_ID = hp.Order_ID
ORDER BY po.Order_ID;
```

Explanation: This query combines multiple tables to generate a complete patient order report containing patient name, medicine name, quantity, order date, total amount, and payment status.

Output:

```
SQL> SELECT
  2      p.Patient_Name,
  3      m.Medicine_Name,
  4      od.Quantity,
  5      po.Order_Date,
  6      po.Total_Amount,
  7      hp.Payment_Status
  8  FROM Patient p
  9  INNER JOIN Patient_Orders po
 10  ON p.Patient_ID = po.Patient_ID
 11  INNER JOIN Order_Details od
 12  ON po.Order_ID = od.Order_ID
 13  INNER JOIN Medicine m
 14  ON od.Medicine_ID = m.Medicine_ID
 15  LEFT JOIN Hospital_Payment hp
 16  ON po.Order_ID = hp.Order_ID
 17  ORDER BY po.Order_ID;

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
Ananya Kumar
Paracetamol
         2 04-OCT-26           13 Success

Ananya Kumar
Amoxicillin
         1 04-OCT-26           13 Success

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
```

```
Rahul Sharma
Cetirizine
         5 04-OCT-26         10.5 Success

Meena Devi
Omeprazole

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_STATUS
---------- --------- ------------ --------------------
         2 04-OCT-26           10 Success

Arun Raj
Azithromycin
         1 04-OCT-26           12 Failed
```

8. PATIENT PURCHASE HISTORY

8.1 Display Medicines Purchased by a Particular Patient

SQL Query:

```
SELECT
    p.Patient_Name,
    po.Order_ID,
    po.Order_Date,
    m.Medicine_Name,
    od.Quantity,
    od.Price
FROM Patient p
INNER JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
INNER JOIN Order_Details od
ON po.Order_ID = od.Order_ID
INNER JOIN Medicine m
ON od.Medicine_ID = m.Medicine_ID
WHERE p.Patient_ID = 301;
```

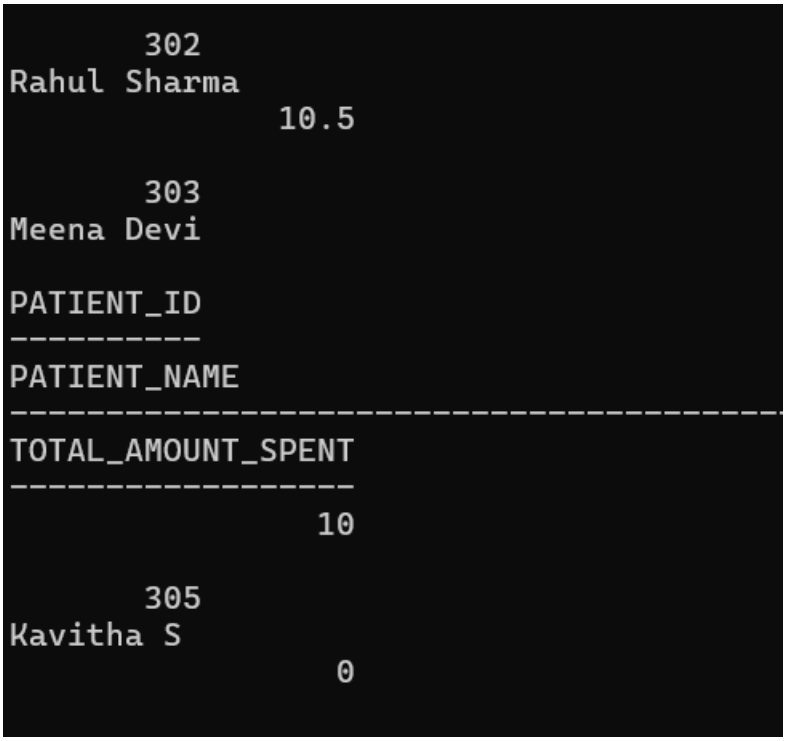
Explanation: This query retrieves the medicine purchase history of the patient whose Patient ID is 301.

Output:

```
SQL> SELECT
  2      p.Patient_Name,
  3      po.Order_ID,
  4      po.Order_Date,
  5      m.Medicine_Name,
  6      od.Quantity,
  7      od.Price
  8  FROM Patient p
  9  INNER JOIN Patient_Orders po
 10  ON p.Patient_ID = po.Patient_ID
 11  INNER JOIN Order_Details od
 12  ON po.Order_ID = od.Order_ID
 13  INNER JOIN Medicine m
 14  ON od.Medicine_ID = m.Medicine_ID
 15  WHERE p.Patient_ID = 301;

PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID ORDER_DAT
---------- ---------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Ananya Kumar
       501 04-OCT-26
Paracetamol
         2        2.5


PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID ORDER_DAT
---------- ---------
MEDICINE_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
Ananya Kumar
       501 04-OCT-26
```



8.2 Calculate Total Amount Spent by Each Patient

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    NVL(SUM(po.Total_Amount), 0)
        AS Total_Amount_Spent
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
ORDER BY Total_Amount_Spent DESC;
```

Explanation: This query calculates the total order amount for each patient. The NVL function replaces NULL totals with zero.

Output:

8.3 Count Orders Placed by Each Patient

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    COUNT(po.Order_ID) AS Number_Of_Orders
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
ORDER BY Number_Of_Orders DESC;
```

Explanation: This query counts the number of orders placed by each patient.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      COUNT(po.Order_ID) AS Number_Of_Orders
  5  FROM Patient p
  6  LEFT JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  GROUP BY p.Patient_ID, p.Patient_Name
  9  ORDER BY Number_Of_Orders DESC;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------
NUMBER_OF_ORDERS
----------------
       304
Arun Raj
                1

       301
Ananya Kumar
                1

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------
NUMBER_OF_ORDERS
----------------

       303
Meena Devi
                1

       302
```

```
       302
Rahul Sharma

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
----------------
              1

       305
Kavitha S
              0
```

8.4 Display Latest Purchase Details of Patients

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    po.Order_ID,
    po.Order_Date AS Latest_Purchase_Date,
    po.Total_Amount,
    po.Order_Status
FROM Patient p
INNER JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
WHERE po.Order_Date = (
    SELECT MAX(po2.Order_Date)
    FROM Patient_Orders po2
    WHERE po2.Patient_ID = p.Patient_ID
)
ORDER BY p.Patient_ID;
```

Explanation: This query retrieves the most recent purchase information for each patient. If multiple orders have the same latest date, they may all appear.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      po.Order_ID,
  5      po.Order_Date AS Latest_Purchase_Date,
  6      po.Total_Amount,
  7      po.Order_Status
  8  FROM Patient p
  9  INNER JOIN Patient_Orders po
 10  ON p.Patient_ID = po.Patient_ID
 11  WHERE po.Order_Date = (
 12      SELECT MAX(po2.Order_Date)
 13      FROM Patient_Orders po2
 14      WHERE po2.Patient_ID = p.Patient_ID
 15  )
 16  ORDER BY p.Patient_ID;

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID LATEST_PU TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------
       301
Ananya Kumar
       501 04-OCT-26           13 Delivered

       302
Rahul Sharma
       502 04-OCT-26         10.5 Pending

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID LATEST_PU TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------

       303
Meena Devi
```

```
       304
Arun Raj

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------------------
  ORDER_ID LATEST_PU TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------
       504 04-OCT-26           12 Pending
```

9. MULTI-TABLE REPORT GENERATION

9.1 Report 1 – Patient Order Report

SQL Query:

```
SELECT
    p.Patient_Name,
    po.Order_ID,
    po.Order_Date,
    po.Order_Status
FROM Patient p
INNER JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
ORDER BY po.Order_ID;
```

Explanation: This report displays patient names, order IDs, order dates, and order statuses.

Output:

```
SQL> SELECT
  2      p.Patient_Name,
  3      po.Order_ID,
  4      po.Order_Date,
  5      po.Order_Status
  6  FROM Patient p
  7  INNER JOIN Patient_Orders po
  8  ON p.Patient_ID = po.Patient_ID
  9  ORDER BY po.Order_ID;

PATIENT_NAME
--------------------------------------------------------------------
  ORDER_ID ORDER_DAT ORDER_STATUS
---------- --------- --------------------
Ananya Kumar
       501 04-OCT-26 Delivered

Rahul Sharma
       502 04-OCT-26 Pending

Meena Devi
       503 04-OCT-26 Pending


PATIENT_NAME
--------------------------------------------------------------------
  ORDER_ID ORDER_DAT ORDER_STATUS
---------- --------- --------------------
Arun Raj
       504 04-OCT-26 Pending
```

9.2 Report 2 – Medicine Sales Report

SQL Query:

```
SELECT
    m.Medicine_Name,
    SUM(od.Quantity) AS Quantity_Sold,
    SUM(od.Quantity * od.Price) AS Total_Revenue
FROM Medicine m
INNER JOIN Order_Details od
ON m.Medicine_ID = od.Medicine_ID
GROUP BY m.Medicine_Name
ORDER BY Total_Revenue DESC;
```

Explanation: This query calculates the total quantity ordered and total sales value for each medicine based on order details.

Output:

```
SQL> SELECT
  2      m.Medicine_Name,
  3      SUM(od.Quantity) AS Quantity_Sold,
  4      SUM(od.Quantity * od.Price) AS Total_Revenue
  5  FROM Medicine m
  6  INNER JOIN Order_Details od
  7  ON m.Medicine_ID = od.Medicine_ID
  8  GROUP BY m.Medicine_Name
  9  ORDER BY Total_Revenue DESC;

MEDICINE_NAME
--------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Cetirizine
            5          17.5

Azithromycin
            1            12

Omeprazole
            2            10


MEDICINE_NAME
--------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
Amoxicillin
            1             8

Paracetamol
            2             5
```

```
            THEN 1
            ELSE 0
        END
    ) AS Successful_Payments
FROM Hospital_Payment
GROUP BY Payment_Mode
ORDER BY Number_Of_Transactions DESC;
```

Explanation: This query analyses payment modes and displays the number of transactions and successful payments for each mode.

Output:

```
SQL> SELECT
  2      Payment_Mode,
  3      COUNT(*) AS Number_Of_Transactions,
  4      SUM(
  5          CASE
  6              WHEN Payment_Status = 'Success'
  7              THEN 1
  8              ELSE 0
  9          END
 10      ) AS Successful_Payments
 11  FROM Hospital_Payment
 12  GROUP BY Payment_Mode
 13  ORDER BY Number_Of_Transactions DESC;

PAYMENT_MODE         NUMBER_OF_TRANSACTIONS SUCCESSFUL_PAYMENTS
-------------------- ---------------------- --------------------
Net Banking                               1                    0
UPI                                       1                    1
Cash                                      1                    1
Card                                      1                    1
```

9.4 Report 4A – Top Purchasing Patients

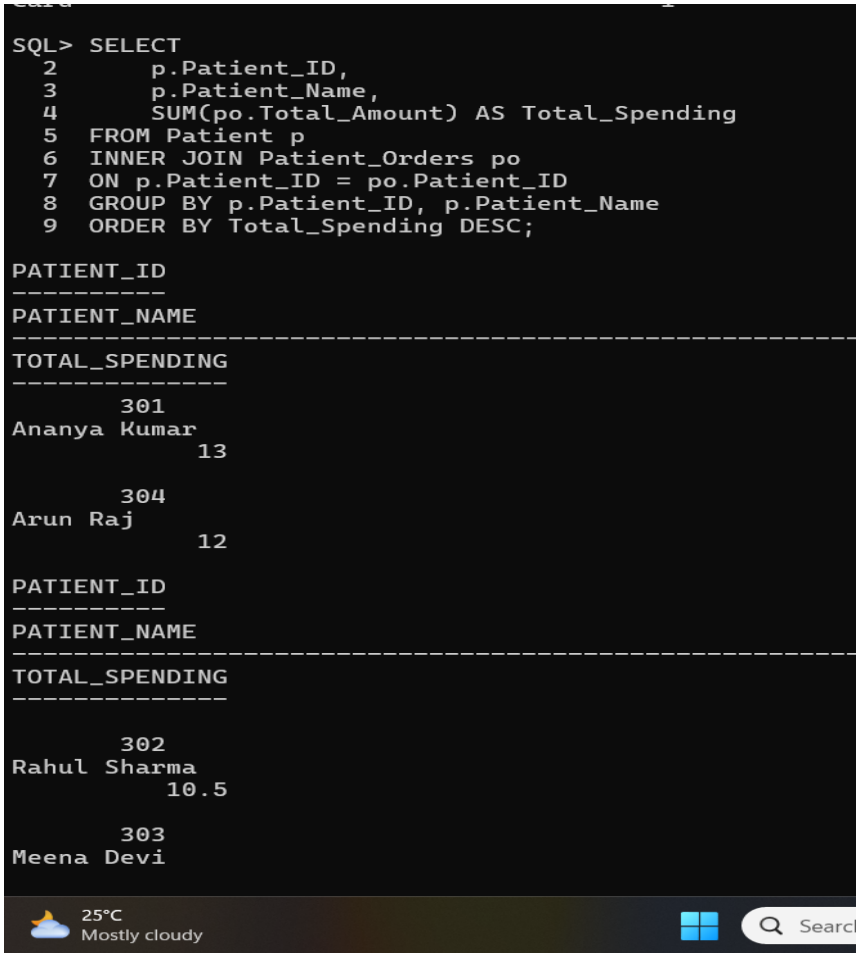
SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    SUM(po.Total_Amount) AS Total_Spending
FROM Patient p
INNER JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
```

```
GROUP BY p.Patient_ID, p.Patient_Name
ORDER BY Total_Spending DESC;
```

Explanation: This query ranks patients according to their total order amounts.

Output:



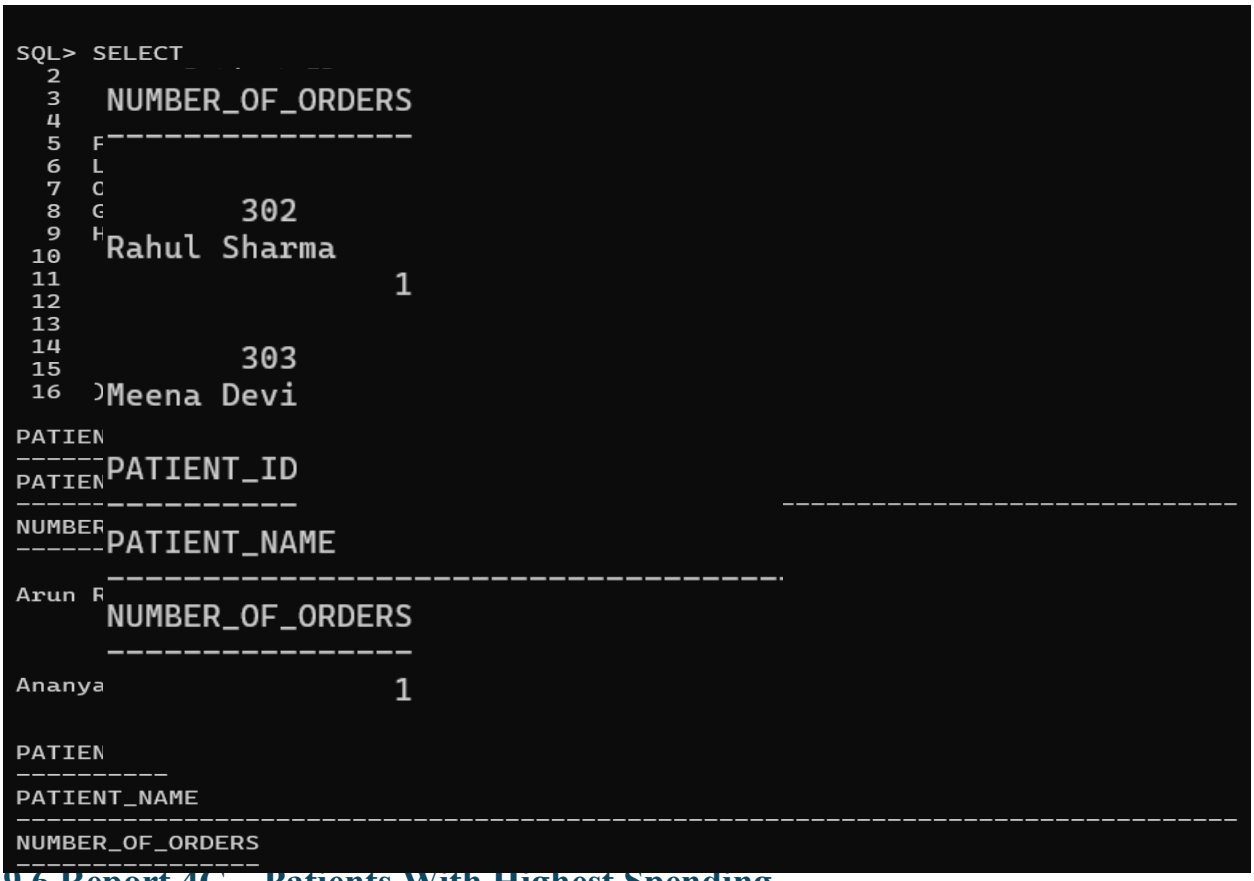
9.5 Report 4B – Patients With Maximum Orders

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    COUNT(po.Order_ID) AS Number_Of_Orders
FROM Patient p
LEFT JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
HAVING COUNT(po.Order_ID) = (
    SELECT MAX(Order_Count)
    FROM (
        SELECT COUNT(*) AS Order_Count
        FROM Patient_Orders
        GROUP BY Patient_ID
    )
);
```

Explanation: This query identifies the patients who have placed the highest number of orders.

Output:



9.6 Report 4C – Patients With Highest Spending

SQL Query:

```
SELECT
    p.Patient_ID,
    p.Patient_Name,
    SUM(po.Total_Amount) AS Total_Spending
FROM Patient p
INNER JOIN Patient_Orders po
ON p.Patient_ID = po.Patient_ID
GROUP BY p.Patient_ID, p.Patient_Name
HAVING SUM(po.Total_Amount) = (
    SELECT MAX(Total_Spending)
    FROM (
        SELECT SUM(Total_Amount) AS Total_Spending
        FROM Patient_Orders
        GROUP BY Patient_ID
    )
);
```

Explanation: This query identifies the patient or patients with the highest total order amount.

Output:

```
SQL> SELECT
  2      p.Patient_ID,
  3      p.Patient_Name,
  4      SUM(po.Total_Amount) AS Total_Spending
  5  FROM Patient p
  6  INNER JOIN Patient_Orders po
  7  ON p.Patient_ID = po.Patient_ID
  8  GROUP BY p.Patient_ID, p.Patient_Name
  9  HAVING SUM(po.Total_Amount) = (
 10      SELECT MAX(Total_Spending)
 11      FROM (
 12          SELECT SUM(Total_Amount) AS Total_Spending
 13          FROM Patient_Orders
 14          GROUP BY Patient_ID
 15      )
 16  );

PATIENT_ID
----------
PATIENT_NAME
--------------------------------------------------------------------
TOTAL_SPENDING
--------------
       301
Ananya Kumar
            13
```

10. RESULT

The Week 8 module demonstrates the implementation of SQL joins and multi-table queries in the CityCare Hospital Management System.

The queries are designed to retrieve patient order details, medicine purchases, payment information, patient purchase history, and analytical reports. The actual results are displayed in the corresponding SQL*Plus screenshots.

11. CONCLUSION

The Database Relationship Analysis module demonstrates how SQL joins can combine information stored in multiple related tables.

INNER JOIN is used to retrieve matching records, LEFT JOIN is used to include all records from the left table, and RIGHT JOIN is used to include all records from the right table.

Multi-table queries and aggregate functions are used to analyse patient purchases, medicine sales, payment transactions, and order history.

Thus, Week 8 helps improve the understanding of relational database operations and demonstrates how meaningful hospital management reports can be generated using Oracle SQL.